

Zu Aufgabe MyStack<T> mit Array

In der Lösung ist die Funktion `hasCapacity()` folgendermaßen implementiert:

```
private boolean hasCapacity() {  
    return elements.length < size;  
}
```

Schauen wir uns an, was passiert, wenn wir einen neuen `MyStack` mit einer Kapazität von 10 erstellen und das erste Element hinzufügen:

```
MyStack<Integer> stack = new MyStack<>(); // INITIAL_CAPACITY = 10  
System.out.println("Length: " + stack.elements.length); //Ausgabe: "Length:  
10"  
System.out.println("Size: " + stack.size); //Ausgabe: "Size: 0"  
  
stack.push(10);  
// Ruft ensureCapacity() auf.  
// Ruft hasCapacity() auf.  
// Der Code prüft: return elements.length < size;  
// Mit unseren Werten: return 10 < 0;  
// Ergebnis: false  
  
System.out.println("Length: " + stack.elements.length); //Ausgabe: "Length:  
20"  
System.out.println("Size: " + stack.size); //Ausgabe: "Size: 1"
```

Da `hasCapacity()` `false` zurückgibt (also "keine Kapazität"), geht `ensureCapacity()` in den `if`-Block und ruft `enlargeArray()` auf.

Das Array wird also verdoppelt, obwohl noch 10 freie Plätze da waren. Beim nächsten `push` (Länge nun 20, Size 1) passiert genau das Gleiche (`20 < 1` ist `false`), und es wird wieder verdoppelt.

```
private boolean hasCapacity() {  
    // Richtig: Es ist Platz, solange die Anzahl der Elemente (size)  
    // kleiner ist als die Länge des Arrays.  
    return size < elements.length;  
}
```

Hier die korrigierte Ausgabe:

```
MyStack<Integer> stack = new MyStack<>(); // INITIAL_CAPACITY = 10  
System.out.println("Length: " + stack.elements.length); //Ausgabe: "Length:  
10"  
System.out.println("Size: " + stack.size); //Ausgabe: "Size: 0"  
  
stack.push(10);  
// Ruft ensureCapacity() auf.  
// Ruft hasCapacity() auf.  
// Der Code prüft: return size < elements.length;  
// Mit unseren Werten: return 0 < 10;  
System.out.println("Kapazität: " + stack.hasCapacity()); // Ausgabe:  
"Kapazität: true"  
  
System.out.println("Length: " + stack.elements.length); //Ausgabe: "Length:  
10"  
System.out.println("Size: " + stack.size); //Ausgabe: "Size: 1"
```

Zusammenfassung:

Mit dem **falschen** Code passiert ein folgendes (exponentielles) Wachstum bei fast leerem Stack:

push(1)	Vergrößert auf:	20
push(2)	Vergrößert auf:	40
push(3)	Vergrößert auf:	80

Mit dem **korrigierten** Code (`size < length`):

push(1)	bleibt bei:	10
push(2)	bleibt bei:	10
...		
push(...)	bleibt bei:	10
...		
push(11)	Vergrößert auf	20

Wie testen wir das schnell und pragmatisch? (In diesem Fall)

Frech für den Test das `private` entfernen:

```
private boolean hasCapacity() {  
    // Richtig: Es ist Platz, solange die Anzahl der Elemente (size)  
    // kleiner ist als die Länge des Arrays.  
    return size < elements.length;  
}
```

Das Ziel von Objektorientierung ist es, Komplexität zu verstecken.

Manchmal heiligt der Zweck die Mittel.

Testbarkeit: Es ist extrem schwer, private Methoden zu testen. Wenn wir das `private` entfernen (das nennt man dann package-private oder "default visibility"), kann dein Test (der im selben Package liegt) die Methode sehen, aber der "Rest der Welt" (andere Packages) nicht.

Das ist ein oft genutzter Kompromiss.

In der professionellen Java-Entwicklung macht man oft Folgendes, wenn man eine Methode testen muss, sie aber eigentlich privat sein sollte:

Man entfernt das `private`, fügt aber eine Annotation oder einen Kommentar hinzu:

```
// Eine eigene Annotation  
public @interface VisibleForTesting {  
}
```

Mit eigener Annotation:

```
@VisibleForTesting // Das 'private' ist weg, aber die Annotation warnt  
andere Entwickler  
boolean hasCapacity() {  
    return size < elements.length;  
}
```

Unser Test:

```
@Test  
void testHasCapacity() {  
    // Arrange  
    MyStack<Integer> stack = new MyStack<>(10);  
  
    // Act & Assert  
    // Jetzt können wir direkt zugreifen  
    assertTrue(stack.hasCapacity(), "Sollte true sein, da Array leer  
ist.");  
}
```

Hier fällt es nun auf, wenn die Kapazität nicht korrekt erkannt wird.